



Bases de Datos II

Bases de Datos NoSQL / Práctica con MongoDB.

Parte I

Sección 1: Bases de Datos NoSQL y Relacionales

Si bien las BBDD NoSQL tienen diferencias fundamentales con los sistemas de BBDD Relacionales o RDBMS, algunos conceptos comunes se pueden relacionar. Responda las siguientes preguntas, considerando MongoDB en particular como Base de Datos NoSQL.

1. ¿Cuáles de los siguientes conceptos de RDBMS existen en MongoDB? En caso de no existir, ¿hay alguna alternativa? ¿Cuál es?
 - Base de Datos
 - Tabla / Relación
 - Fila / Tupla
 - Columna
2. Existen claves foráneas en MongoDB? ¿Qué diferencias existen con las bases de datos de tipo relacional?
3. Para acelerar las consultas, MongoDB tiene soporte para índices. ¿Qué tipos de índices soporta?
4. En MongoDB existen dos tipos de vistas, explique brevemente cuáles son y qué diferencias existen entre ellas. Además mencione algunos casos donde podría utilizarlas.
5. Los documentos de una colección pueden diferir en la cantidad y sus tipos de campos, ¿existen algunas formas de validar los elementos a insertar en una colección para evitar esta disparidad?
6. MongoDB tiene soporte para transacciones, pero no es igual que el de los RDBMS. ¿Cuál es el alcance de una transacción en MongoDB?
7. Las relaciones entre documentos en MongoDB pueden establecerse mediante documentos embebidos o referencias. Investigue cómo se implementa cada una y analice las ventajas y desventajas de cada una, comparándola con la forma estándar de establecer relaciones en una base de datos relacional.
8. Tomando como referencia el modelo de los trabajos prácticos anteriores y suponiendo que este podría mapearse a una base de datos en MongoDB, proponga algunos casos



donde la relación sería conveniente ser mapeada como referencia y otros como documentos embebidos. Justifique la elección.

Sección 2: Operaciones CRUD básicas

Descargue la última versión de MongoDB desde el sitio oficial. Ingrese al cliente de línea de comando para realizar los siguientes ejercicios.

9. Cree una nueva base de datos llamada “tours”, y una colección llamada “recorridos”.
 - i. En esta nueva colección y utilizando el comando correspondiente inserte un nuevo documento (un recorrido) con los siguientes atributos:

```
{ "nombre": "City Tour", "precio": 200, "stops": ["Diagonal Norte", "Avenida de Mayo", "Plaza del Congreso"], "totalKm": 5 }
```
 - ii. Recupere la información del producto usando el comando `db.products.find()` (puede agregar la función `.pretty()` al final de la expresión para ver los datos indentados). Notará que no se encuentran exactamente los atributos que insertó. ¿Cuál es la diferencia?
10. Agregue a la colección creada en el punto anterior, utilizando un solo comando, los documentos especificados en el documento “material_adicional_1.json”. Sobre ellos realice las siguientes operaciones:
 - i. Actualizar el recorrido “Cultural Odyssey” para que su total de kilómetros sea 12.
 - ii. Actualizar el listado de Stops del recorrido con nombre “Delta Tour” para agregar “Tigre”.
 - iii. Aumentar un 10% el precio de todos los recorridos.
 - iv. Eliminar el recorrido con nombre “Temporal Route”
 - v. Crear el array de etiquetas (tags) para la ruta “Urban Exploration”. Agregue el elemento “Gastronomía” a dicho arreglo.
11. Sobre la colección generada en el punto anterior realice las siguientes consultas utilizando la función `find()`
 - i. Obtenga la ruta con nombre “Museum Tour”
 - ii. Las rutas con precio superior a \$600
 - iii. Las rutas con precio superior a \$500 y con un total de kilómetros mayor a 10.
 - iv. Las rutas que incluyan el stop “San Telmo”
 - v. Las rutas que incluyan el stop “Recoleta” y no el stop “Plaza Italia”
 - vi. El nombre y el total de km (si es que posee) de las rutas que incluyan el stop “Delta” y tenga un precio menor a 500
 - vii. Las rutas que incluyen tanto “San Telmo” como “Recoleta” y “Avenida de Mayo” entre sus stops.
 - viii. Solo el nombre de las rutas que dispongan de más de 5 stops
 - ix. Las rutas que no tengan definida el total de sus kilómetros.
 - x. Los nombres y el listado de stops de aquellas rutas que incluyen algún museo en sus recorridos.
 - xi. Obtenga la cantidad de elementos que posee la colección.



Sección 3: Aggregation Framework

Aggregation Framework es una herramienta de MongoDB que nos permite ampliar la posibilidad de uso de su lenguaje de consulta. Así, podemos realizar consultas complejas, filtrado, agrupación y análisis de datos en tiempo real, utilizando una serie de etapas de procesamiento que se aplican secuencialmente a los documentos en una colección.

12. Cree una nueva base de datos llamada “tours2”. Guarde el archivo llamado ‘generador1.js’ adjunto a esta práctica y ejecútelo con: `load(<ruta del archivo ‘generador1.js’)`. Si utiliza un cliente que lo permita (ej. Robo3T), se puede ejecutar directamente en el espacio de consultas. Examine las colecciones generadas.
13. Utilizando Aggregation Framework, realice las siguientes operaciones:
 - i. Obtenga una muestra de 5 rutas aleatorias de la colección
 - ii. Extienda la consulta anterior para incluir en el resultado toda la información de cada una de las Stops. Note que puede ligarlas por su código.
 - iii. Obtenga la información de las Routes (incluyendo la de sus Stops) que tengan un precio mayor o igual a 900
 - iv. Obtenga la información de las Routes que tengan 5 Stops o más.
 - v. Obtenga la información de las Routes que tengan incluido en su nombre el string “111”.
 - vi. Obtenga solo las Stops de la Route con nombre “Route100”
 - vii. Obtenga la información del Stop que más apariciones tiene en Routes.
 - viii. Obtenga las Route que tengan un precio inferior a 150. A ellos agregué una nueva propiedad que especifique la cantidad de Stops que posee la Route. Cree una nueva colección llamada “rutas_economicas” y almacene estos elementos.
 - ix. Por cada Stop existente en su colección, calcule el precio promedio de las Routes que la incluyen

Tener en cuenta: si la consulta se empieza a tornar difícil de leer, se pueden ir guardando los agregadores en variables, que no son más que objetos en formato JSON.

Fecha de cierre de esta etapa

6 de junio